WEB TECHNOLOGIES I / ASSIGNMENT 04

Introduction to JavaScript

Implementation report | Steam Game Hub | 21 October 2026

Team Eggs · Group SE-2507

Ташкенбаев Нурсултан

Бигали Абулхаир

Султанкулов Шынгыс

Лопаткин Артем

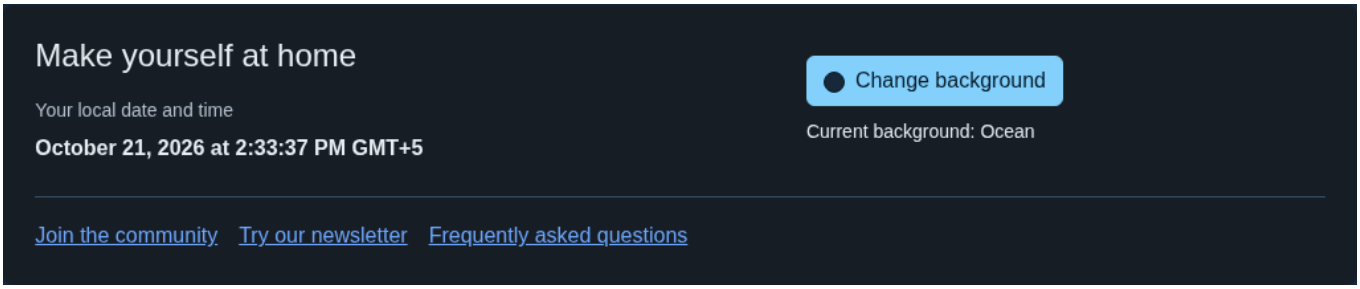
Objective

Extend the existing responsive game hub with five JavaScript interactions: form validation, a FAQ accordion, a subscription popup, a background colour switcher, and a live date/time display. Preserve the existing navigation, galleries and page styling.

Published website

https://abilkhayrbigali-design.github.io/

The public GitHub Pages version was checked on 21 October 2026. Its four updated HTML pages, JavaScript and stylesheet match the tested local source files.



Home-page controls on the published website.

Deliverable	Contents
Website	Nine HTML pages, local assets, shared JavaScript and CSS.
Report + ZIP	Task steps, screenshots, testing evidence, deployment URL and project files.

PART 1 / TASK 1 / 15%

Validate form input

Location: community.html#join-community | Shared logic: js/site.js

01. Add labelled fields for a display name, email address, password and password confirmation. Mark all four as required; set the password minimum to 8 characters.

02. Attach a submit listener. Use **preventDefault()** to keep the demo local, then check blank values, email format, password length and matching confirmation.

03. Show an error next to each invalid field, connect it with **aria-describedby**, mark **aria-invalid** and focus the first error.

04. Re-check edited fields after a failed submission, including confirmation when the password changes. Clear form resets both values and error messages.

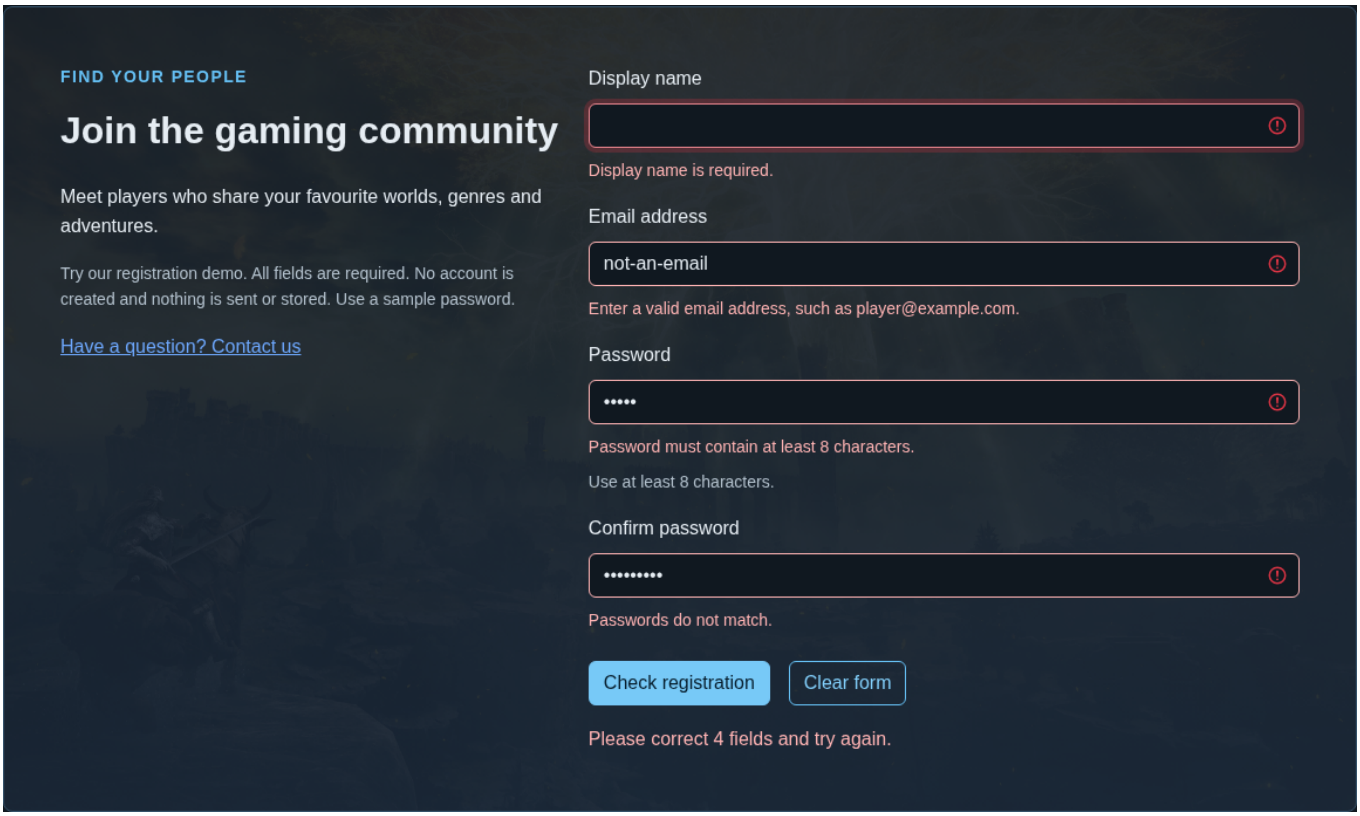


Figure 1. Invalid name, email, password length and confirmation each produce a specific message.

Verified: empty and whitespace-only values are rejected; valid details produce a success message. Editing a previously valid password re-checks its confirmation. Contact and newsletter forms use the same validation code.

The 8-character minimum is the project rule. Forms do not create accounts or send/store entered data.

PART 1 / TASK 2 / 15%

Reveal FAQ answers

Location: contact.html#faq | Four independent questions

01. Create each question as a button with **aria-controls** pointing to its answer panel and **aria-expanded="false"** as the initial state.

02. Hide answers with a CSS grid row of **0fr** and make their contents inert so hidden links cannot receive keyboard focus.

03. On click, toggle the expanded state, the **is-open** class and the panel's inert property. The CSS grid expands to **1fr** over 0.25 seconds.

04. Use native buttons so Enter and Space also toggle answers. Disable animation when the user requests reduced motion.

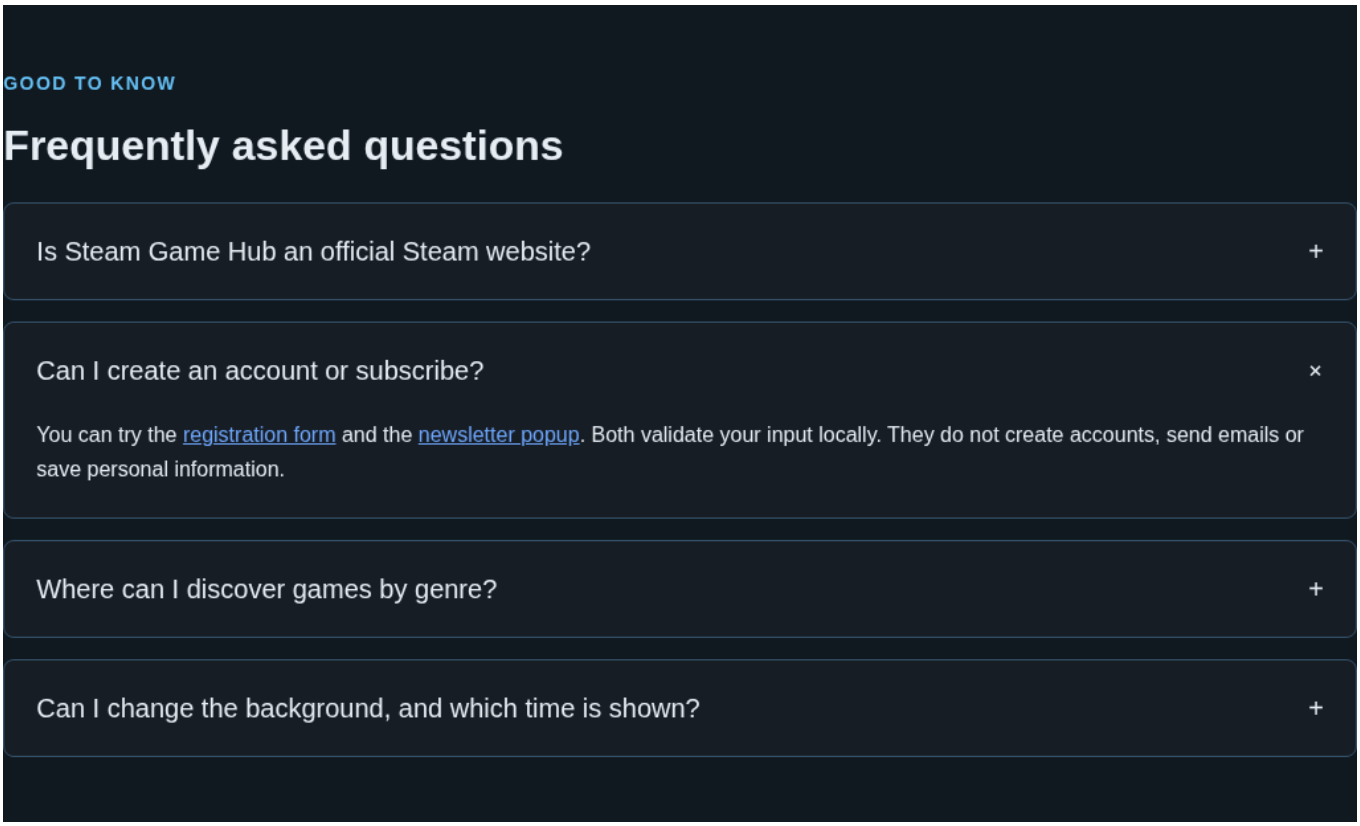


Figure 2. The account/subscription answer is expanded; the other answers remain collapsed.

Verified: all answers start hidden. Every question opens and closes by pointer and keyboard. Links become available only while their answer is open. Multiple questions can remain open at once.

PART 1 / TASK 3 / 10%

Open a subscription popup

Location: news.html#newsletter | Native HTML dialog

01. Keep the subscription form inside a **<dialog>** element without the open attribute, so it is initially hidden.

02. Connect Subscribe to updates to **showModal()**. Focus moves to the email field and the background page becomes inert.

03. Close the popup with its close button, a click outside its bounds or Escape. Keep Tab and Shift+Tab cycling through the popup controls.

04. Validate the email using the shared form handler, show the result locally, and restore focus to the opener when the dialog closes.

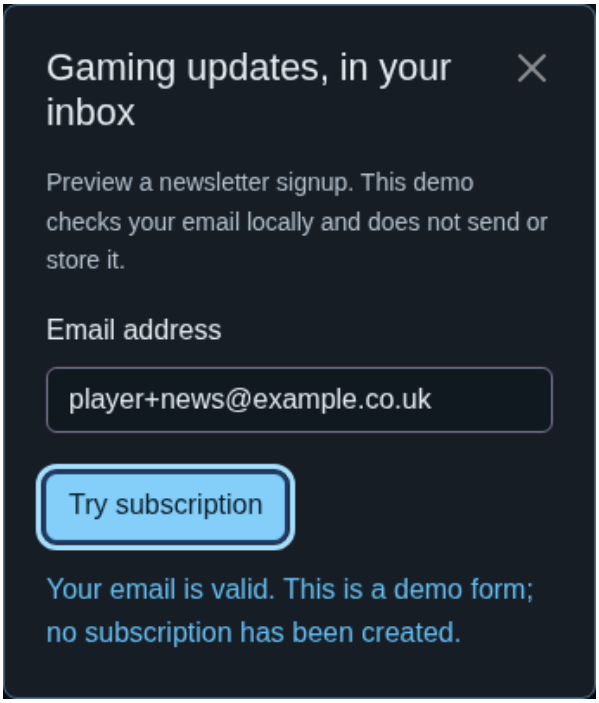


Figure 3. The popup after a valid email submission at a 375 px viewport.

Interaction checks

Initially hidden
Opens from its button
Rejects empty/invalid email
Accepts valid email
Closes via button, backdrop and Escape
Returns keyboard focus
Fits the mobile viewport

Demo behaviour

The success message confirms valid input. It does not claim a real subscription.

PART 1 / TASK 4 / 5%

Change the page background

Location: index.html#hub-tools | Four-colour cycle

01. Define an array of four named colours: Midnight, Ocean, Plum and Forest.

02. Listen for a click on Change background and advance the current index with **(index + 1) % colours.length**.

03. Assign the selected colour to **document.body.style.backgroundColor**. Update the visible colour name and the button swatch.

04. Use a 0.3-second CSS transition for a smooth change while retaining readable text contrast.

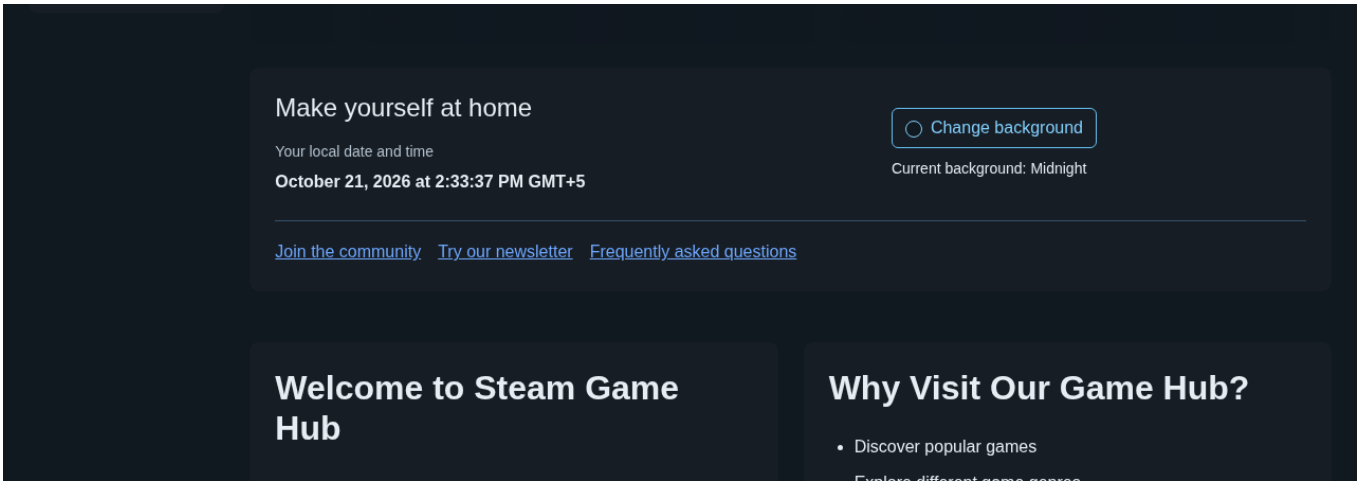


Figure 4a. Midnight: initial background (#101820).

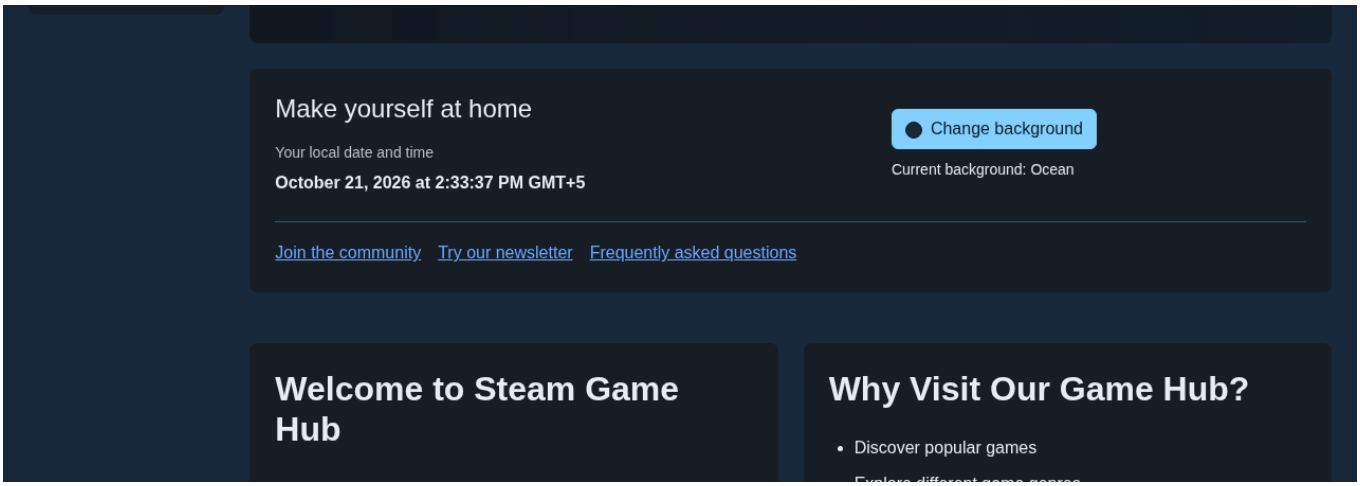


Figure 4b. Ocean: background after one click (#17283b).

Verified: four consecutive clicks produce Ocean, Plum, Forest and Midnight, returning to the original colour. The computed body colour was checked in both browsers at all three viewport widths.

PART 1 / TASK 5 / 5%

Display the current date and time

Location: index.html#hub-tools | Local device timezone

01. Create a semantic **<time>** element and obtain the current instant with **new Date()**.

02. Format the date, year, hours, minutes, seconds and timezone using **Intl.DateTimeFormat**. Store an ISO timestamp in the datetime attribute.

03. Render immediately, update every 1,000 milliseconds with **setInterval**, and refresh when the browser tab becomes visible again.

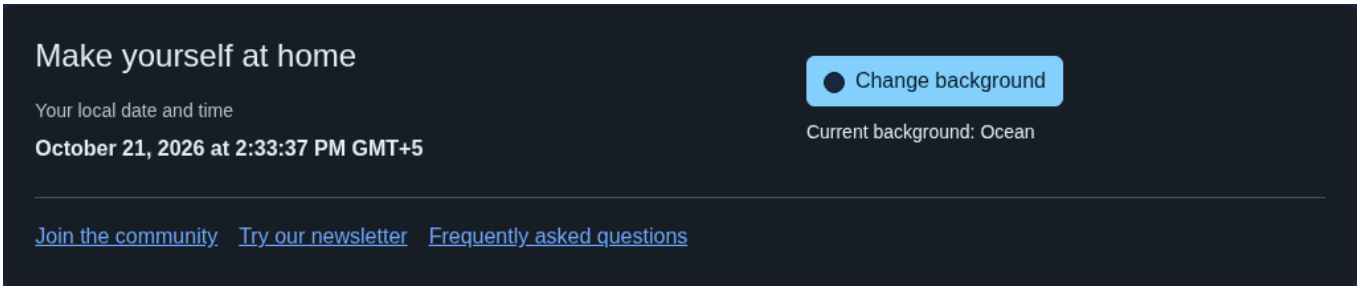


Figure 5. Readable date and time on the deployed home page, including seconds and the local timezone.

Key implementation

```
const now = new Date();
clock.dateTime = now.toISOString();
clock.textContent = formatter.format(now);
setInterval(updateClock, 1000);
```

Boundary verification

Scenario	Observed result
Normal browsing	The displayed timestamp changes automatically on the next second.
Simulated year boundary	Asia/Almaty: December 31, 2026, 11:59:59 PM advances to January 1, 2027, 12:00:01 AM after two simulated seconds.
Direct local opening	The live clock also works when index.html is opened via file://.

The clock follows the device timezone. The constantly changing time is not placed in a live announcement region, avoiding repeated screen-reader announcements.

PART 2 / DEPLOYMENT

Publish and verify the website

GitHub Pages | Source branch: main | Implementation commit: 5c80c5c

Public URL

https://abilkhayrbigali-design.github.io/

Repository

github.com/abilkhayrbigali-design/abilkhayrbigali-design.github.io

01. Save the updated HTML, CSS, JavaScript and test files in the project repository. Commit the changes on main.

02. Push main to the shared GitHub repository. GitHub Pages serves the updated static website at the public URL above.

03. Open the public site and exercise all five interactions. Compare the downloaded source files with the local tested files to confirm the deployed version.

Repeat the publishing workflow

```
git add .
git commit -m "Complete Assignment 4 JavaScript features"
git push origin main
```

Deployment evidence

Check	Result
Four updated pages	index.html, community.html, contact.html and news.html: HTTP 200, exact SHA-256 match with local files.
Shared assets	js/site.js and css/theme.css: HTTP 200, exact SHA-256 match.
Public interactions	Clock, background, registration, FAQ and popup all passed the live browser check.
Evidence files	report/assignment4/deployment-verification.json and live-browser-verification.json.

Verified on 21 October 2026. The public site needs no local server or build step. All site libraries and images are included in the project.

QUALITY ASSURANCE

Test the complete experience

Two browser engines | Three viewport widths | Nine pages

Browser	Viewports	Result
Chromium 154.0.8037.97	375, 768, 1440 px	51 / 51 checks passed
Firefox 153.0	375, 768, 1440 px	51 / 51 checks passed
Published site (Chromium)	1280 px	Five live feature checks passed

What was checked

• All nine pages load without horizontal overflow or broken images; their footers retain the team names.

• Forms handle missing values, whitespace-only values, invalid email, short passwords, mismatched confirmation, correction after submission and reset.

• FAQ questions toggle through mouse and keyboard. The popup opens, keeps focus within its controls and closes in all three supported ways.

• The colour cycle returns to the original colour; the clock updates and crosses a simulated year boundary correctly.

• Existing mobile navigation and carousel behaviour still work. Reduced-motion settings and direct file opening are supported.

• No JavaScript errors or unintended form submissions were observed. Local/session storage remained empty during form interactions.

Reproduce the checks

```
npm install
npx playwright install chromium firefox
npm test
BROWSER_ENGINE=firefox npm test
```

The site itself requires no package installation. These commands install and run developer test tools only. Test scenarios are in tests/assignment4.cjs; full results and component screenshots are in report/assignment4/.

REFLECTION AND DEMONSTRATION

Explain the implementation

Preparation for the practice-lesson defense

Final reflection

The five interactions connect events, DOM state and CSS presentation. Reusing one form handler keeps validation consistent across registration, contact and subscription. Native buttons and dialog behaviour provide a useful starting point for keyboard access; explicit focus handling and meaningful errors complete the interaction.

Testing exposed a keyboard-focus issue in the popup, which was corrected by cycling Tab and Shift+Tab through its controls. Testing on small and large screens confirmed that the original navigation and carousels still work. The implementation remains a static academic demonstration: real registration, message delivery and subscriptions would require a server and server-side validation.

Questions to prepare

Question	Answer to demonstrate
Why preventDefault()?	It stops native form submission so JavaScript can validate fields and display a local result.
Why check confirmation again?	Changing the original password can invalidate a confirmation that previously matched.
How does the accordion work?	The click handler changes aria-expanded, the is-open class and inert; CSS animates the grid row height.
Why use showModal()?	It opens the dialog as a modal and makes the background page inert. Escape, close controls and focus are handled as part of the interaction.
Why use modulo for colours?	The remainder operation wraps the index back to zero after the last palette entry.
Why create a new Date each tick?	Each update reads the current device time instead of accumulating a manually incremented counter.

Practice sequence

Submit invalid registration data, fix it, toggle a FAQ answer, open and close the popup, cycle the background and show the ticking clock. Be ready to locate the corresponding event listener and explain its state changes.

Each team member must be familiar with the team implementation and complete the required individual contribution. Each member submits the report, ZIP and website URL before the deadline shown in LMS, then defends the work in the practice lesson.